

```
=====
crypto-reused-iv.md
=====
```

Crypto: Reused Nonce (AES-CTR)

Summary

Recover the flag by exploiting AES-CTR nonce reuse, which reuses the same keystream for every encryption.

Challenge

- The server encrypts user-provided plaintext via POST /encrypt.
- It also returns a target ciphertext via GET /target (the flag encrypted).
- Vulnerability: AES-CTR is used with a fixed nonce (FIXED_NONCE) for all requests.

Solution

In CTR mode:

- $C = P \oplus \text{keystream}$

If the same nonce is reused, the keystream repeats. If we can ask the server to encrypt a known plaintext (like all-zero bytes), we directly learn the keystream:

- $\text{encrypt}(00..00) = \text{keystream}$

Then decrypt:

- $P = C \oplus \text{keystream}$

Reproduction

1. Run the challenge: `python challenges/crypto-reused-iv/server.py` (listens on `http://localhost:5003`).
2. Run the exploit: `python exploits/exploit_reused_iv.py`.

Flag

flag{never_reuse_iv}

Mitigation

- Never reuse a CTR nonce for the same key.
- Prefer AEAD constructions (AES-GCM or ChaCha20-Poly1305) with correct nonce handling.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only. Do not use these techniques on systems you do not own or have explicit permission to test.

```
=====
forensics-hidden-pdf.md
=====
```

Forensics: Hidden in PDF

Summary

Extract an embedded attachment (flag.txt) from a PDF using either standard CLI tools or a Python script.

Challenge

- File: `challenges/forensics-hidden-pdf/challenge.pdf`

- The flag is embedded as an attachment in the PDF name tree (/EmbeddedFiles).

Solution

Approach

1. Inspect the PDF for embedded files.
2. Extract attachments and look for a flag string.

Tooling options

- Poppler tools (Linux/macOS): `pdftdetach -list` and `pdftdetach -saveall`
- Included Python extractor: `exploits/extract_pdf_flag.py`

Reproduction

1. (Optional) Regenerate the PDF: `python challenges/forensics-hidden-pdf/create_challenge.py`
2. Extract attachments: `python exploits/extract_pdf_flag.py challenges/forensics-hidden-pdf/challenge.pdf`

Flag

flag{pdf_attachment_found}

Mitigation

- Strip attachments from untrusted PDFs before redistribution.
- Validate/sanitize PDFs in ingestion pipelines.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only
not use these techniques on systems you do not own or have explicit permission to test.

=====

pwn-off-by-one-rop.md

=====

Pwn: Off-by-One ROP

Summary

Abuse a 1-byte overflow to corrupt the least significant byte (LSB) of saved RBP, then pivot the stack into attacker-controlled data and return into `win()`.

Challenge

- Bug: `read(STDIN, buf, 129)` reads up to 129 bytes into a 128-byte buffer.
- Impact: byte index 128 overwrites the first byte of saved RBP (on x86-64, little-endian).
- Mitigations: stack protector is disabled here (`-fno-stack-protector`) to keep focus on the pivot.

Solution

Why the LSB matters

If we can nudge saved RBP to point into our buffer, the function epilogue (`leave; ret`) effectively does:

- `leave`: `RSP = RBP`; `RBP = [RSP]`
- `ret`: `RIP = [RSP+8]`

So if RSP is pivoted into our buffer, we control the return target.

Steps

1. Place a fake stack frame at the start of `buf`:
 - fake RBP (8 bytes)
 - fake RIP (8 bytes) set to `win()`
2. Send 129 bytes so the final byte overwrites the LSB of saved RBP and points it into `buf`.
3. Let the function return; execution reaches `win()`.

Reproduction

This challenge is intended for Linux (or WSL).

1. Build: `make -C challenges/pwn-off-by-one-rop/`
2. Run the exploit: `python exploits/exploit_off_by_one.py`

Flag

`flag{off_by_one_stack_pivot}`

Mitigation

- Use correct bounds (`read(..., sizeof(buf))`) and always check return values.
- Keep hardening on (PIE, RELRO, stack protector) in real applications; it raises the bar substantially.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only not use these techniques on systems you do not own or have explicit permission to test.

=====
pwn-stack-canary-slip.md
=====

Pwn: Stack Canary Slip

Summary

Leak a stack canary via a format string bug, then reuse it to bypass stack protector and redirect execution to `win()`.

Challenge

- Vulnerabilities (in `vuln()`):
 - `printf(buf)` format string (stack leak)
 - `gets(buf)` buffer overflow
- Mitigation present: stack canary (`-fstack-protector`)
- Win condition: call `win()` which prints `flag{canary_leak_then_smash}` and exits.

Solution

High level

1. Use the format string to leak the canary value from the stack.
2. Overflow the buffer again, but include the exact canary bytes so the check passes.
3. Overwrite the saved return address with the address of `win()`.

Notes on offsets

The correct `%n$` offset for the canary is environment- and build-dependent. The intended workflow is to fuzz offsets until you see a value that ends with `00` (canaries typically have null least significant byte).

Reproduction

This challenge is intended for Linux (or WSL).

1. Build: `make -C challenges/pwn-stack-canary/`
2. Run the exploit: `python exploits/exploit_stack_canary.py`

Flag

`flag{canary_leak_then_smash}`

Mitigation

- Never pass user input as a format string: use `printf("%s", buf)`.
- Avoid unsafe input APIs like `gets()`: use `fgets()` with bounds.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only
not use these techniques on systems you do not own or have explicit permission to test.

=====
rev-flag-check.md
=====

Reversing: Flag Check

Summary

Recover the expected flag by reversing a simple XOR-based check that compares input bytes
against static arrays.

Challenge

The checker validates:

`ord(flag[i]) XOR KEY[i] == TARGET[i]`

Both KEY and TARGET are static, so the flag can be derived offline:

`flag[i] = KEY[i] XOR TARGET[i]`

Solution

Steps

1. Extract KEY and TARGET from the binary (or, in this repo, from flag_check.py).
2. Compute each flag byte as `KEY[i] XOR TARGET[i]`.
3. Print the reconstructed string.

Reproduction

1. Run the solver: `python exploits/solve_flag_check.py`
2. (Optional) Verify: `python challenges/rev-flag-check/flag_check.py` and paste the output.

Flag

`flag{xor_is_not_crypto}`

Mitigation

- Do not embed secrets in client-side binaries if the attacker controls execution.
- Move validation and secrets server-side, or use a design that remains secure under revers engineering.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only
not use these techniques on systems you do not own or have explicit permission to test.

=====
web-cookie-smith.md
=====

Web: Cookie Smith

Summary

Forge the auth cookie by abusing a SHA-1 length extension weakness in a naive "secret-prefi
signature.

Challenge

- Endpoint: `GET /profile?user=guest` sets `auth=<base64(...)>`

- Cookie format (decoded): user|role|sha1(SECRET || "user|role")
- Protected page: GET /admin returns the flag if admin is present in the parsed fields.

Solution

The server signs SHA1(SECRET || message). Because SHA-1 is Merkle-Damgard, knowing the hash of an unknown-prefix message allows computing the hash of:

SECRET || message || glue_padding || extra

without knowing SECRET.

Steps

1. Fetch a valid cookie via GET /profile.
2. Base64-decode to split message and sig.
3. Perform SHA-1 length extension to append |admin to the message.
4. Re-encode the forged cookie and request GET /admin with it.

Reproduction

1. Run the challenge (Python + Flask): python challenges/web-cookie-smith/app.py (listens on http://localhost:5001).
2. Run the exploit: python exploits/exploit_cookie_smith.py.

Flag

flag{length_extension_broke_auth}

Mitigation

- Replace secret-prefix hashing with an HMAC: HMAC-SHA256(secret, message).
- Do not store authorization decisions (like role) in client-controlled data unless it is strongly authenticated and validated.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only. Do not use these techniques on systems you do not own or have explicit permission to test.

```
=====
web-sqli-search.md
=====
```

Web: SQLi in Search

Summary

Exploit a UNION-based SQL injection in a SQLite-backed search endpoint to read the flags table.

Challenge

- Endpoint: GET /search?q=<input>
- Vulnerable query shape:
SELECT title FROM posts WHERE title LIKE '%<q>%'
- App also echoes SQLite exceptions to the client (useful for discovery).

Solution

Because q is interpolated into SQL without parameters, we can terminate the string and append a UNION SELECT that returns the flag.

Steps

1. Confirm injection: q=' OR 1=1 --' should return all posts.
2. Because the original query selects 1 column (title), use a 1-column UNION payload.
3. Extract flag: q=' UNION SELECT flag FROM flags --'

Reproduction

1. Run the challenge: `python challenges/web-sqli-search/app.py` (listens on `http://localhost:5002`).
2. Run the exploit: `python exploits/exploit_sqli_search.py`.

Flag

`flag{union_select_ftw}`

Mitigation

- Use parameterized queries (prepared statements), including for LIKE patterns.
- Do not return raw DB exceptions to users; log them server-side.

Ethics note: this writeup targets the intentionally vulnerable code in this repository only not use these techniques on systems you do not own or have explicit permission to test.